

Attribution Circuits to the Refusal Direction

Part 1: The Mathematics

1.1 Review: Standard Attribution Graphs (Ameisen et al. 2025)

In Anthropic's circuit tracing framework, the model's MLP layers are replaced by a **cross-layer transcoder (CLT)** whose features are sparsely active and more interpretable. The key objects are:

CLT features. At layer ℓ , feature activations are:

$$a^\ell = \text{JumpReLU}\left(W_{enc}^\ell x^\ell\right)$$

where x^ℓ is the residual stream at layer ℓ .

Local replacement model. For a fixed prompt p , the local replacement model: 1. Substitutes CLT features for MLP layers, 2. Freezes attention patterns and LayerNorm denominators from the underlying model's forward pass on p , 3. Adds an error correction at each (position, layer) so that activations exactly match the underlying model.

After freezing, the entire local replacement model is piecewise-linear (the only nonlinearities are JumpReLU thresholds on feature pre-activations). This means feature-to-feature interactions are **linear**, and attribution is well-defined.

Edge weights. Given source feature s (layer ℓ_s , position c_s) and target feature t (layer ℓ_t , position c_t), the edge weight is:

$$A_{s \rightarrow t} = a_s \cdot w_{s \rightarrow t}$$

where a_s is the activation of s and $w_{s \rightarrow t}$ is the virtual weight. The virtual weight decomposes via a stop-gradient Jacobian:

$$w_{s \rightarrow t} = \sum_{\ell_s \leq \ell < \ell_t} \left(W_{dec,s}^{\ell_s \rightarrow \ell}\right)^T J_{c_s, \ell \rightarrow c_t, \ell_t}^\bullet W_{enc,t}^{\ell_t}$$

where: - $W_{dec,s}^{\ell_s \rightarrow \ell}$ is the decoder vector of feature s writing to layer ℓ , - $W_{enc,t}^{\ell_t}$ is the encoder vector of feature t , - $J^\bullet$ is the Jacobian with stop-gradients on all nonlinearities (MLPs, attention patterns, LayerNorm denominators).

Output attribution. For attributing to output logits, the target node is a logit z_v for token v . The edge from feature s to logit z_v is computed analogously, replacing the encoder of the target with the unembedding vector $W_U[:, v]$.

The pre-activation of any feature (or the logit) is the **sum of its incoming edges**, giving a complete linear decomposition.

1.2 Review: The Refusal Direction (Arditi et al. 2024)

Arditi et al. showed that refusal behavior in chat models is mediated by a single direction $\hat{r}$ in the residual stream. This direction is found via **difference-in-means**:

$$\hat{r}^{(\ell)} = E_{\text{harmful}}[x^{(\ell)}] - E_{\text{harmless}}[x^{(\ell)}]$$

computed at each layer ℓ and a chosen token position (typically the last prompt token). The final refusal direction $\hat{r}$ is selected from the layer with the strongest separation, then normalized: $\hat{r} \leftarrow \hat{r} / \|\hat{r}\|$.

Key causal findings: - **Ablation**: erasing $\hat{r}$ from the residual stream at all layers prevents refusal on harmful prompts. - **Addition**: adding $\hat{r}$ to the residual stream elicits refusal even on harmless prompts.

The refusal “metric” at layer ℓ , position c , is the scalar projection:

$$R^{(\ell, c)} = \langle x^{(\ell, c)}, \hat{r} \rangle$$

1.3 Derived: Attribution to the Refusal Direction

Now here is the core mathematical contribution of your idea. Instead of attributing to output logits $z_v = x^{(L, c_{\text{target}})} \cdot W_U[:, v]$, we attribute to the **refusal projection**:

$$R = \langle x^{(\ell^*, c^*)}, \hat{r} \rangle$$

where ℓ^* is the chosen measurement layer and c^* is the token position (e.g., last prompt token).

Why this is straightforward. In the local replacement model, the residual stream at (ℓ^*, c^*) is a linear function of all upstream feature activations (because we’ve frozen attention patterns and LayerNorm). And the projection onto $\hat{r}$ is itself a linear functional. So R is a linear function of all upstream feature activations, exactly like a logit.

Concretely, the attribution edge from source feature s to the refusal scalar R is:

$$A_{s \rightarrow R} = a_s \cdot w_{s \rightarrow R}$$

where the virtual weight to the refusal direction is:

$$w_{s \rightarrow R} = \sum_{\ell_s \leq \ell^* < \ell^*} \left(W_{dec,s}^{\ell_s \rightarrow \ell^*} \right)^T J_{c_s, \ell \rightarrow c^*, \ell^*}^{\bullet} \hat{r}$$

This is mathematically identical to logit attribution, with $\hat{r}$ replacing $W_u[:, v]$ as the “readout vector.”

Similarly, for the embedding of input token s :

$$w_{s \rightarrow R} = Emb_s^T J_{c_s, 0 \rightarrow c^*, \ell^*}^{\bullet} \hat{r}$$

And error node attributions follow the same pattern.

The resulting graph has the same structure as a standard attribution graph: - **Input nodes**: token embeddings - **Intermediate nodes**: active CLT features at each (position, layer) - **Output node**: the scalar R (refusal projection), replacing the logit nodes

Edges still decompose linearly: $R = \sum_s A_{s \rightarrow R}$ for all paths.

Generalization. This extends to *any* linear functional on the residual stream, not just the refusal direction. You could attribute to a linear probe direction, a steering vector, an SAE feature’s encoder direction, etc. The math is identical: swap out the readout vector at the output node.

1.4 Practical Considerations

Choice of measurement point (ℓ^*, c^*) . Arditi et al. found the refusal direction is most concentrated in middle-to-late layers (roughly layers 14-20 for 32-layer models). You should pick ℓ^* where the refusal direction has the strongest causal effect. Note: if you measure at $\ell^* < L$, you only attribute through features at layers $< \ell^*$, which may be desirable (you see *how* the refusal direction is built up) but means you miss later contributions.

Partial vs. full attribution. If you attribute at the final layer L , you get the full attribution of everything contributing to R . If you attribute at an intermediate layer ℓ^* , you trace only the “upstream” computation that builds the refusal signal, which is likely more informative for understanding *how the model decides to refuse*.

LayerNorm interaction. The standard framework freezes LayerNorm denominators. Since $R = \hat{r}^T x^{(\ell^*)}$ reads from the residual stream *before* LayerNorm, this is clean. If you instead wanted to measure after LayerNorm (as some probe setups do), you’d need the frozen LayerNorm denominator, but this is already handled by the local replacement model.

Part 2: Related Work

2.1 Directly Related: “Finding Features Causally Upstream of Refusal” (Paleka, Chlenski, Ardit; MATS 2024)

This LessWrong post by Daniel Paleka, Eric Chlenski, and Andy Ardit is the **closest existing work** to your idea. They:

1. Define a refusal metric R as the projection onto the refusal direction at layer 15 (for Gemma-2-2B IT).
2. Compute $\nabla_x R$ (the gradient of R w.r.t. earlier-layer activations) at layer 5 across all token positions.
3. Use SAE features at layer 5 to find features with high “relative gradient” (gradient projected onto each SAE feature’s decoder direction, scaled by the feature’s activation).
4. Verify causality by steering with these upstream features and checking if refusal behavior changes.

Key difference from your proposal: They use standard SAE features and raw gradients, **not** the CLT-based replacement model and attribution graph framework from Anthropic’s circuit tracing. Your idea would produce a *full graph* showing the chain of intermediate computational steps from input tokens through CLT features to the refusal direction, whereas Paleka et al. only compute a single-hop gradient from layer 5 features to the refusal direction at layer 15.

2.2 Sparse Feature Circuits (Marks et al. 2024)

Marks et al. compute attribution graphs using SAE features (not transcoders). Their “attribution patching” computes the linear approximation of each SAE feature’s causal effect on a chosen metric. Although their examples use logit-difference metrics (e.g., “basketball” vs. “golf” logit difference), their framework is fully general: you can plug in **any differentiable scalar metric**, including a residual stream projection. The Anthropic March 2024 update demonstrates this explicitly for the Michael Jordan / sport example, computing $attr_i = a_i d_i \cdot \nabla_x L$ where L is a logit difference.

2.3 “A Single Direction of Truth” (Hallucination Probe Attribution, 2025)

This paper finds a single linear direction in the residual stream that separates hallucinated from faithful text, then uses **gradient-times-activation attribution** to trace which MLP layers contribute most to this direction. While not using the full CLT attribution graph machinery, it demonstrates the same principle: attributing to a linear probe direction rather than to output logits.

2.4 Activation/Attribution Patching to Custom Metrics

The general framework of attribution patching (Syed et al. 2024, Nanda 2022, Kramár et al. 2024) already supports arbitrary scalar loss functions L . The innovation in your proposal is not the ability to target a custom metric (that’s standard), but rather to do it **within the CLT replacement model framework**, producing a full multi-hop attribution graph with interpretable intermediate features, pruning, and visualization. This is a meaningful advance over the single-hop gradient approach.

2.5 Summary Table of Related Work

Work	Method	Target	Multi-hop?
Anthropic Circuit Tracing (2025)	CLT + attribution graph	Output logits	Yes
Paleka et al. (2024)	SAE + gradient of refusal direction	Refusal direction	No (single hop)
Marks et al. (2024)	SAE + attribution patching	Logit difference (but generalizable)	Yes (SAE features)
“Single Direction of Truth” (2025)	Linear probe + grad × act	Hallucination direction	No (per-layer)
Your proposal	CLT + attribution graph	Refusal direction	Yes (full graph)

Part 3: Modifying the circuit-tracer Library

3.1 Library Architecture Overview

The circuit-tracer library (<https://github.com/safety-research/circuit-tracer>) has this structure:

```
circuit_tracer/  
├── replacement_model.py    # ReplacementModel class (wraps  
HookedTransformer or nnsight)  
├── attribution.py           # Core attribution graph computation  
├── graph.py                # Graph data structure, pruning  
├── cli.py                  # Command-line interface  
├── visualization/          # Graph visualization server  
└── ...
```

The main workflow is:

1. **ReplacementModel.from_pretrained(model, transcoder_set)** loads a model and its pre-trained transcoders.
2. **attribute(model, prompt, ...)** runs a forward pass, identifies active features, then computes backward Jacobians to get all edge weights. The backward pass computes gradients from the *output logits* through the frozen replacement model back to each feature's decoder.
3. **prune(graph, ...)** removes low-influence nodes/edges.
4. **Visualization** renders the graph as JSON for the frontend.

3.2 What Needs to Change

The key function to modify is in **attribution.py**, where the backward pass target is defined. Currently, it attributes from the **top-k logit nodes** (those capturing 95% of probability mass, up to 10 tokens). The change is to replace this target.

Step 1: Define a custom target node.

Instead of creating output nodes for each logit $z_v = x^{(L)} \cdot W_U[:, v]$, create a single output node for the refusal projection $R = x^{(\ell^*, c^*)} \cdot \hat{r}$.

In practice, this means: - Load the refusal direction $\hat{r}$ (a vector in $R^{d_{model}}$), pre-computed via the difference-in-means method. - Choose the measurement layer ℓ^* and position c^* . - In the backward pass, instead of computing $\frac{\partial z_v}{\partial features}$ for each logit v , compute $\frac{\partial R}{\partial features}$.

Step 2: Modify the backward Jacobian computation.

The current code computes the stop-gradient Jacobian from the final residual stream through the unembedding. For refusal direction attribution: - The “unembedding” equivalent is simply the vector $\hat{r}$ (instead of a column of W_U). - The Jacobian should be taken from the residual stream at layer ℓ^* , position c^* , not from the logits at layer L .

This is a small change: in the backward pass, instead of seeding the gradient at the logit layer with $W_U[:, v]$, seed it at layer ℓ^* with $\hat{r}$.

Step 3: Modify graph construction and pruning.

- Replace the logit output nodes with a single “refusal_projection” node.
- The pruning algorithm (which traces backward from output nodes to identify important features) works identically since it only cares about edge magnitudes.
- The visualization frontend should label this node differently (e.g., “Refusal Direction” instead of a token name).

3.3 Minimal Code Changes (Pseudocode)

```
# In attribution.py, the core backward pass currently does something like:
# for each target logit v:
#     grad = backprop from logit z_v through frozen model
#     edge_weights[s -> v] = feature_activation[s] * grad[s]

# MODIFIED VERSION:
def attribute_to_direction(model, prompt, direction, layer, position,
**kwargs):
    """
    Attribute to a specific direction in the residual stream.

    Args:
        model: ReplacementModel with loaded transcoders
        prompt: input string
        direction: torch.Tensor of shape (d_model,), the target direction
        (e.g. refusal direction)
        layer: int, which layer's residual stream to measure
        position: int, which token position to measure
    """
    # 1. Forward pass: get all feature activations, attention patterns, etc.
    # (identical to standard attribution)
    activations, cache = model.forward(prompt, return_cache=True)

    # 2. Backward pass: seed gradient at (layer, position) with the direction
    vector
    # instead of seeding at logits with unembedding columns
    #
    # Conceptually:  $R = \text{residual\_stream}[\text{layer}, \text{position}] @ \text{direction}$ 
    #  $dR/d(\text{residual\_stream}[\text{layer}, \text{position}]) = \text{direction}$ 
    #
    # Then backprop through frozen attention + CLT decoders with
    stop-gradients

    # 3. Compute edge weights as activation * gradient for each feature
    # 4. Build and return the attribution graph
```

3.4 Concrete Implementation Steps

1. **Add a direction parameter** to the main attribute() function (or create a new attribute_to_direction() function).
2. **In the backward pass setup**, check if direction is provided. If so:
 - Skip the logit node creation (no need for max_n_logits or desired_logit_prob).
 - Create a single output node labeled by the direction name.
 - Seed the backward pass at the specified layer and position with the direction vector.

3. **Ensure the Jacobian computation handles intermediate layers.** The current code likely computes Jacobians from the final layer. You need to support starting the backward pass from an intermediate layer ℓ^* . This may require extracting the residual stream at ℓ^* as a “virtual output” and backpropping from there.
4. **Graph pruning** should work without changes since it operates on edge weights generically.
5. **Visualization** needs a minor update: label the output node as “Refusal Direction” (or whatever the target direction represents) and display R ’s value instead of logit probabilities.

3.5 Expected Challenges

- **Transcoder availability for chat models.** The refusal direction is only meaningful in instruction-tuned (chat) models. Currently, transcoders are trained on base models (Gemma-2-2B, Llama-3.2-1B). The circuit-tracer demos show that base model transcoders can be applied to instruction-tuned models (`gemma_it_demo.ipynb`), but reconstruction quality may degrade. This is a practical limitation, not a mathematical one.
- **Choosing ℓ^* .** If ℓ^* is too early, you miss important late-layer contributions. If it’s at the final layer, you get everything but the graph may be harder to interpret. A good starting point is the layer where Arditi et al. found the strongest refusal signal for your specific model.
- **Scale of the graph.** Attributing to a single scalar (the refusal projection) rather than multiple logits may actually produce *simpler* graphs, since there’s only one output node to trace back from.
- **Validation.** You should validate the refusal direction graph the same way Anthropic validates standard graphs: by perturbing identified features and checking that the refusal projection R changes as the graph predicts.